

Homework Report

Anton Henkel, Alessandro Comencini, Erik Soehner

September 29, 2026

Abstract

This report examines the prediction of passenger survival in Kaggle's Titanic machine learning competition [1]. Although chance played a role in survival, passenger characteristics may help explain differences in outcomes. The available predictors include ticket class, sex, age, the combined count of siblings and spouses aboard, the combined count of parents and children aboard, ticket number, fare, cabin number, and port of embarkation. The data contain continuous, categorical, and binary variables, with some entries missing. Feature representation and the treatment of missing data are therefore part of the modelling problem: they affect both the relationships the model can learn and how it handles incomplete passenger records. A random forest classifier was selected using stratified 10-fold cross-validation on the labelled training data. The selected model achieved a mean cross-validation accuracy of 83.73% and an accuracy of 78.23% on Kaggle's external test set. We also discuss potential improvements through feature engineering, a more thorough treatment of missing information, and alternative representations and classifiers.

1 Approach

Survival may depend on thresholds and interactions between passenger characteristics. For example, the effect of age could vary with sex and passenger class. Decision trees can capture these nonlinear relationships and interactions through successive splits, without requiring their form to be specified in advance. They require no feature scaling and make few assumptions about the distributions of the predictors, making them a natural starting point for this small dataset with mixed variable types.

A single tree can be sensitive to the particular training sample. We therefore used a random forest to average predictions across trees trained with randomisation, reducing this variability. Tree depth and minimum leaf size provide further control over model complexity. The forest offers a flexible baseline, although the ensemble is less directly interpretable than a single tree and still requires decisions about how to represent categorical variables and handle missing data.

1.1 Model Setup and Training

`train.csv` was used for training, validation, and model selection through stratified 10-fold cross-validation. In each iteration, one fold was held out for validation and the remaining folds were used for fitting. Stratification kept the proportions of survivors and non-survivors approximately the same across folds. The labels in `test.csv` are hidden, so this file serves as the external test set and is evaluated through a competition submission.

2 Results

The selected random forest achieved a mean cross-validation accuracy of 83.73% on the labelled training data and an accuracy of 78.23% on Kaggle's external test set.

For the current submission, the decision to leave missing values without imputation was guided by Kaggle scores. The reported accuracy is therefore not a fully independent assessment of that decision.

3 Potential Directions

3.1 Leveraging Features

The unused `Name`, `Ticket`, and `Cabin` fields offer opportunities to extend the current model. The `Name` field provides titles such as Mr., Miss, or Master. Shared tickets could identify groups of travellers, while cabin information could indicate a passenger's deck or location within the ship. Family size and an indicator of travelling alone can also be derived from `SibSp` and `Parch`. The usefulness of these features may depend on how they relate to the existing predictors: a title might complement age and sex, while deck information might add detail to passenger class. Indicators of missing data could also preserve information about which passenger details were recorded.

3.2 Missing Data and Latent Representations

A direct extension would be to estimate missing values from other passenger attributes. Multi-variate imputation, including tree-based methods, could use relationships that median imputation ignores. Comparing this approach with the forest's native handling of missing values would help establish whether explicitly reconstructing missing information improves prediction.

Probabilistic models offer another option by representing several plausible values for each missing observation. Possible approaches include mixed-family vine copulas and Dirichlet-process mixtures of elliptical copulas; the latter have been proposed specifically for imputing mixed-type data [5]. These models could be linked to the existing classifier by generating plausible completed records and averaging the resulting predictions. This would allow uncertainty about missing features to influence the final prediction.

This also suggests learning a compact representation directly from incomplete records. A latent-class model or a small generalised linear latent-variable model (GLLVM), with observation distributions appropriate to each feature type, could capture shared structure among the predictors [2].

3.3 Classification Models

The random forest could itself provide a new representation by encoding the leaves that each passenger reaches [3]. Logistic regression or a support vector machine (SVM) could then use these features, combining the interactions learned by the forest with a different classification rule. CatBoost would offer a related extension through boosting and native handling of categorical variables. A GAM or spline-based logistic regression would provide a more structured comparison by modeling age and fare as flexible nonlinear functions.

For probabilistic latent representations, an SVM could instead use an expected radial basis function (RBF) kernel that averages similarities over passengers' latent distributions. This would allow the classifier to use the uncertainty retained in the representation, linking the preceding ideas through a kernel defined on distributions [4].

4 Ai use

For this report ai was used to find and quote different studies and was also used to adjust the styling and grammatical corrections of the report.

References

- [1] Kaggle. Titanic: Machine learning from disaster. <https://www.kaggle.com/competitions/titanic>. Accessed September 28, 2026.
- [2] Jenni Niku, Wesley Brooks, Riki Herliansyah, Francis K. C. Hui, Sara Taskinen, and David I. Warton. Efficient estimation of generalized linear latent variable models. *PLOS ONE*, 14(5):e0216129, 2019. <https://doi.org/10.1371/journal.pone.0216129>.
- [3] scikit-learn developers. Feature transformations with ensembles of trees. https://scikit-learn.org/stable/auto_examples/ensemble/plot_feature_transformation.html.
- [4] Bharath K. Sriperumbudur, Arthur Gretton, Kenji Fukumizu, Bernhard Schölkopf, and Gert R. G. Lanckriet. Hilbert space embeddings and metrics on probability measures. *Journal of Machine Learning Research*, 11:1517–1561, 2010. <https://www.jmlr.org/papers/v11/sriperumbudur10a.html>.
- [5] Jiali Wang, Anton Westveld, Bronwyn Loong, and Alan Welsh. An imputation model by Dirichlet process mixture of elliptical copulas for data of mixed type. arXiv:1910.05473. <https://arxiv.org/abs/1910.05473>, 2019.

A Python Code

A.1 Model Training and Submission

```
1 import pandas as pd
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5
6
7 # Preprocessing and feature selection
8 train = pd.read_csv("titanic/data/train.csv")
9 test = pd.read_csv("titanic/data/test.csv")
10
11 features = [
12     "Pclass", # passenger class (1st, 2nd, 3rd)
13     "Sex",
14     "Age",
15     "SibSp", # number of siblings/spouses aboard
16     "Parch", # number of parents/children aboard
17     "Fare", # price of the ticket
18     "Embarked" # boarding location (C, Q, S)
19 ]
20 X = train[features].copy()
21 y = train["Survived"]
22 X_test = test[features].copy()
23
24 # one-hot encoding
25 X = pd.get_dummies(X, columns=["Sex", "Embarked"])
26 X_test = pd.get_dummies(X_test, columns=["Sex", "Embarked"])
27
28 # Align columns of test set with training set
29 X_test = X_test.reindex(columns=X.columns, fill_value=0)
30
31
32 # -----
33 # Random Forest with tuned hyperparameters (random_forest_titanic_hyparams.py)
34
35 model = RandomForestClassifier(
36     n_estimators=150,
37     max_depth=None,
38     min_samples_split=15,
39     min_samples_leaf=2,
40     max_features="sqrt",
41     random_state=1
42 )
43 # Note: Tuning without imputation produced the same Kaggle accuracy despite different hyperparameters.
44
45 # Validation
46 X_train, X_val, y_train, y_val = train_test_split(
47     X,
48     y,
49     test_size=0.2,
50     random_state=0,
51     stratify=y
52 )
53
54 model.fit(X_train, y_train)
55 val_predictions = model.predict(X_val)
56 validation_accuracy = accuracy_score(y_val, val_predictions)
57 print("Validation accuracy:", validation_accuracy)
58
```

```
59 |
60 | # Train the model on the entire training set
61 | model.fit(X, y)
62 |
63 |
64 | # Predict on test set
65 | test_predictions = model.predict(X_test)
66 |
67 |
68 | # Create Kaggle submission
69 | submission = pd.DataFrame({
70 |     "PassengerId": test["PassengerId"],
71 |     "Survived": test_predictions
72 | })
73 |
74 | submission.to_csv(
75 |     "titanic/submission_rf_titanic_no_imputation.csv",
76 |     index=False
77 | )
78 |
79 | print("Submission file created.")
```

A.2 Hyperparameter Tuning

```
1 import pandas as pd
2 from sklearn.model_selection import StratifiedKFold, GridSearchCV
3 from sklearn.ensemble import RandomForestClassifier
4
5 """
6 Hyperparameter Tuning for Random Forest on the Titanic Dataset
7 """
8
9 # Preprocessing and feature selection
10 train = pd.read_csv("titanic/data/train.csv")
11 test = pd.read_csv("titanic/data/test.csv")
12
13 features = [
14     "Pclass", # passenger class (1st, 2nd, 3rd)
15     "Sex",
16     "Age",
17     "SibSp", # number of siblings/spouses aboard
18     "Parch", # number of parents/children aboard
19     "Fare", # price of the ticket
20     "Embarked" # boarding location (C, Q, S)
21 ]
22 X = train[features].copy()
23 y = train["Survived"]
24 X_test = test[features].copy()
25
26
27 # one-hot encoding
28 X = pd.get_dummies(X, columns=["Sex", "Embarked"])
29 X_test = pd.get_dummies(X_test, columns=["Sex", "Embarked"])
30
31 # Align columns of test set with training set
32 X_test = X_test.reindex(columns=X.columns, fill_value=0)
33
34
35 # 10-fold cross-validation for hyperparameter tuning
36 cv = StratifiedKFold(
37     n_splits=10,
38     shuffle=True,
39     random_state=0
40 )
41
42 # Random Forest
43 model = RandomForestClassifier(
44     random_state=1
45 )
46
47 # Hyperparameter combinations to test
48 param_grid1 = {
49     "n_estimators": [100, 200, 500],
50     "max_depth": [None, 5, 10, 15],
51     "min_samples_split": [2, 5, 10],
52     "min_samples_leaf": [1, 2, 4],
53     "max_features": ["sqrt", "log2"]
54 }
55 """
56 Best paramters from param_grid1:
57 Best parameters:
58 {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, '
59     n_estimators': 200}
60 CV accuracy: 0.8350187265917602
61 """
```

```

61 # following up after param_grid1 to refine the search space
62 param_grid2 = {
63     "n_estimators": [150, 200, 300, 400],
64     "max_depth": [None],
65     "min_samples_split": [10, 15, 20],
66     "min_samples_leaf": [2, 3, 4],
67     "max_features": ["sqrt"]
68 }
69 """
70 Best paramters from param_grid2:
71 {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 15, '
    n_estimators': 150}
72 CV accuracy: 0.8372784019975029
73 """
74
75 # Grid Search
76 grid_search = GridSearchCV(
77     estimator=model,
78     param_grid=param_grid2,
79     cv=cv,
80     scoring="accuracy",
81     n_jobs=-1,
82     verbose=1
83 )
84
85 grid_search.fit(X, y)
86
87 print("Best parameters:")
88 print(grid_search.best_params_)
89
90 print("\nBest CV accuracy:")
91 print(grid_search.best_score_)

```